



Universidad Internacional del Ecuador

LÓGICA DE PROGRAMACIÓN

Tema: PROYECTO INTEGRADOR

Yessenia Elizabeth Lasso Gordon

1A

Lilian Marlene Aman Ramos

Junio, 2026

1. Desarrollo del Tema: Generador Seguro de Contraseñas

¿Qué es y para qué sirve el sistema?

El Generador Seguro de Contraseñas es un programa desarrollado en Python que permite al usuario crear, de forma automática, contraseñas aleatorias y de alta seguridad. El usuario define dos parámetros: la longitud de la contraseña y los tipos de caracteres que desea incluir. El sistema procesa esos parámetros y devuelve una contraseña lista para usarse.

La necesidad de este tipo de herramienta es concreta: la mayoría de las personas utiliza contraseñas débiles o reutilizadas porque inventar una combinación verdaderamente aleatoria de letras, números y símbolos es difícil de hacer mentalmente. El programa resuelve ese problema en menos de un segundo.

Funcionalidades del sistema

N.º	Funcionalidad	Descripción
RF-01	Ingresar longitud	El usuario define cuántos caracteres tendrá la contraseña (mínimo 8, máximo 64).
RF-02	Seleccionar tipos de caracteres	Permite elegir entre mayúsculas (A-Z), minúsculas (a-z), dígitos (0-9) y símbolos especiales (!@#\$%^&*).
RF-03	Generar contraseña	El sistema combina aleatoriamente los caracteres seleccionados y produce la contraseña.
RF-04	Mostrar resultado	La contraseña generada se muestra en pantalla de forma clara y legible.
RF-05	Copiar al portapapeles	El usuario puede copiar la contraseña con un solo clic para usarla de inmediato.

Diagrama de Caso de Uso

El siguiente diagrama muestra cómo interactúa el usuario con el sistema. El actor principal (Usuario) realiza cuatro acciones: ingresar los parámetros, solicitar la generación, ver la contraseña y copiarla. Las relaciones «include» indican pasos que ocurren siempre; la relación «extend» indica un paso opcional.

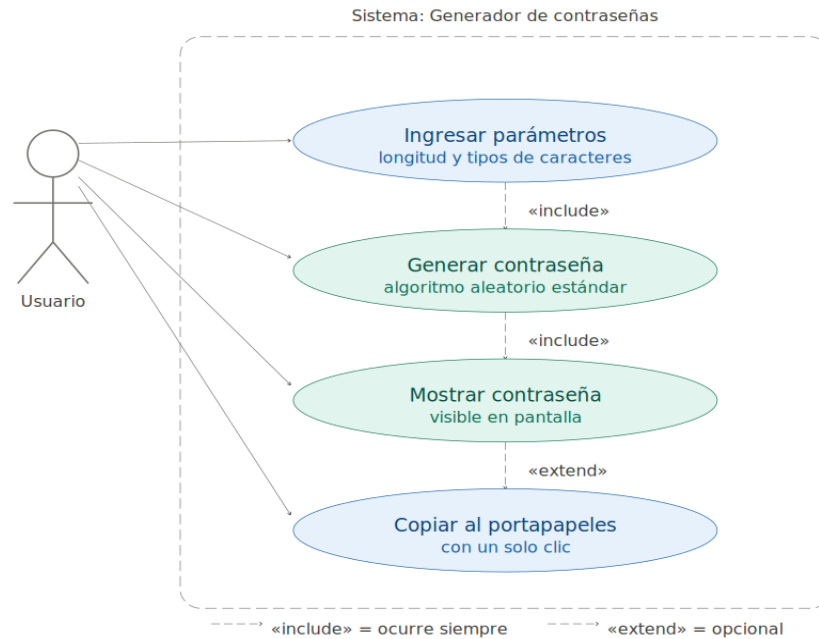


Figura 1. Diagrama de Caso de Uso — Generador Seguro de Contraseñas

Diagrama de Flujo (RAPTOR)

El diagrama de flujo representa paso a paso la lógica interna del algoritmo de generación de contraseñas. Fue elaborado en la herramienta RAPTOR Avalonia y muestra con precisión el orden en que el programa toma decisiones y ejecuta acciones.

El flujo inicia solicitando la longitud dentro de un bucle de validación que repite la solicitud hasta obtener un valor válido (mayor o igual a 8). Luego pregunta al usuario qué tipos de caracteres desea incluir, verificando que se haya seleccionado al menos dos opciones. Mediante condicionales anidados construye el conjunto de caracteres disponibles agregando solo los grupos activados. Finalmente, un bucle recorre la longitud solicitada y en cada iteración selecciona aleatoriamente un carácter del conjunto para ir formando la contraseña, que se muestra al terminar.

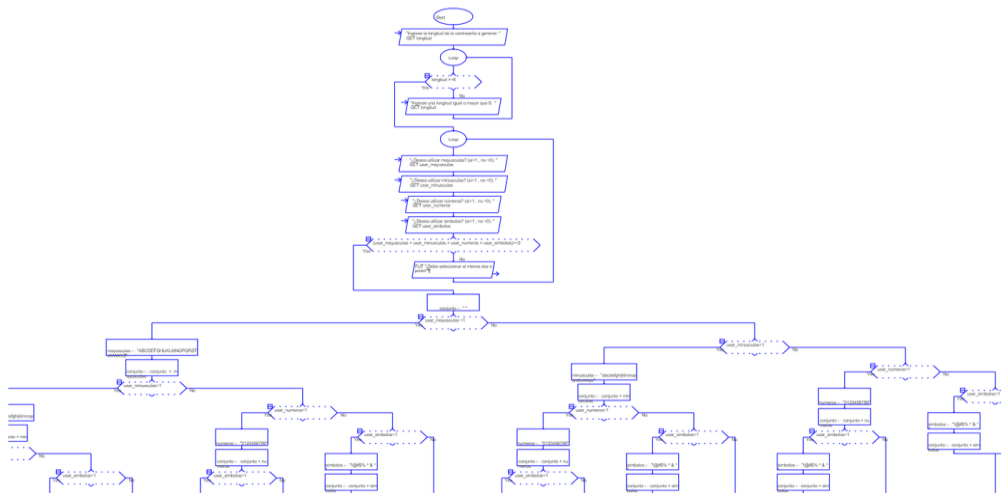


Figura 2. Diagrama de Flujo elaborado en RAPTOR Avalonia — Generador Seguro de Contraseñas

2. Análisis del Sistema: El Porqué del Proyecto

Identificación del problema

En la actualidad, cada persona administra en promedio más de 80 cuentas digitales, cada una protegida por una contraseña. Sin embargo, el comportamiento más común es el uso de contraseñas cortas, predecibles o reutilizadas, como fechas de nacimiento, nombres de mascotas o secuencias simples como "123456". Este hábito representa una de las principales causas de vulneraciones de seguridad a nivel mundial.

El problema no es falta de conciencia, sino falta de herramientas accesibles. Crear manualmente una combinación verdaderamente aleatoria de caracteres es cognitivamente difícil y poco práctico en el día a día. Allí es donde el software puede intervenir de forma directa.

Justificación de la solución

Un generador de contraseñas automatiza exactamente lo que el ser humano hace mal: la selección aleatoria de caracteres sin ningún patrón. Al delegar esta tarea al programa, se eliminan los sesgos cognitivos y se garantiza que el resultado sea impredecible para cualquier atacante.

Adicionalmente, la solución es accesible, no requiere conexión a internet, no almacena ningún dato y puede ejecutarse en cualquier computadora con Python instalado. Esto la hace útil tanto para usuarios individuales como en entornos organizacionales básicos.

Relación con las nuevas tecnologías y la sociedad

Este proyecto se enmarca directamente en el impacto que las tecnologías de la información tienen sobre la seguridad personal. La ciberseguridad ya no es un tema exclusivo de grandes empresas, cada usuario con una cuenta en línea es un potencial objetivo. El desarrollo de herramientas simples que protejan a las personas representa un uso responsable y de alto valor de la programación.

Más allá del código, el proyecto refleja un principio fundamental: la tecnología más valiosa no es siempre la más compleja, sino la que resuelve con precisión un problema real de las personas.

Arquitectura del sistema (tres capas)

Capa	Responsabilidad	Qué contiene
Presentación	Interacción directa con el usuario.	Campo de longitud, checkboxes, botones Generar y Copiar, mensajes de validación.

Lógica de negocio	Procesamiento: validación y generación.	Funciones construir_conjunto(), validar_parametros(), generar_contrasena().
Datos	Provisión de los conjuntos de caracteres.	Cadenas de mayúsculas, minúsculas, dígitos y símbolos especiales.

3. Resultados Obtenidos

¿Qué produce el programa?

Al ejecutar el programa, el usuario obtiene una contraseña completamente aleatoria cuya longitud y composición de caracteres respetan exactamente los parámetros que definió. Algunos ejemplos de salidas del sistema:

Longitud	Tipos seleccionados	Ejemplo de resultado	Nivel de seguridad
8	Mayúsculas + números	K7T2M9PQ	Moderado
12	Todos los tipos	aB3#kZ9!mR1@	Alto
16	Todos los tipos	Xw\$4jP!8nQ@2rL#7	Muy alto
6	Solo minúsculas	qzrtpm	Débil (no recomendado)

Validaciones en acción

El programa no solo genera contraseñas, también verifica que los parámetros sean válidos antes de ejecutar la generación:

- Si el usuario no selecciona ningún tipo de carácter, el programa muestra un mensaje de advertencia y no genera nada.
- Si la longitud ingresada está fuera del rango permitido (menor a 8 o mayor a 64), el programa lo notifica sin cerrarse.
- Si todos los parámetros son válidos, la generación ocurre en menos de un segundo.

Diagramas entregados como parte del proyecto

- Diagrama de Caso de Uso (UML) — muestra las interacciones entre el usuario y el sistema.
- Diagrama de Flujo (RAPTOR) — representa paso a paso la lógica del algoritmo de generación.
- Diagrama de Capas — muestra la arquitectura interna del sistema en tres niveles.

Evaluación general del sistema

El programa cumple con todos los requerimientos funcionales definidos en la fase de análisis. La contraseña se genera correctamente, los mensajes de error aparecen cuando

corresponde, y el código está organizado en funciones con responsabilidad única. El sistema fue versionado en GitHub con commits progresivos que documentan la evolución del desarrollo.

4. Implementación y Complicaciones

Cómo funciona el código

La implementación en Python se organizó en tres bloques funcionales bien diferenciados, siguiendo la arquitectura de tres capas definida en el diseño:

Bloque 1 — Construcción del conjunto de caracteres

Dependiendo de las opciones que el usuario activó, se construye una tupla que contiene únicamente los caracteres permitidos. Se usan estructuras condicionales (if) para agregar cada grupo:

```
# Tuplas con los grupos de caracteres
mayusculas = ("A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M",
              "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z")

minusculas = ("a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m",
              "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z")

numeros = ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9")

simbolos = ("!", "@", "#", "$", "%", "^", "&", "*")
```

```
conjunto = ()
if usar_mayusculas == 1:
    conjunto = conjunto + mayusculas
if usar_minusculas == 1:
    conjunto = conjunto + minusculas
if usar_numeros == 1:
    conjunto = conjunto + numeros
if usar_simbolos == 1:
    conjunto = conjunto + simbolos
```

Bloque 2 — Generación carácter por carácter

Una vez construido el conjunto, se recorre con un bucle for tantas veces como la longitud solicitada. En cada iteración se genera un índice aleatorio y se toma el carácter de esa posición:

```
for i in range(longitud):
    indice = random.randint(0, len(conjunto) - 1)
    contrasena = contrasena + conjunto[indice]
```

Bloque 3 — Validación de parámetros

Antes de ejecutar la generación, el programa verifica que el conjunto no esté vacío y que la longitud sea válida. Si alguna condición falla, muestra un mensaje al usuario:

```
longitud = int(input("Ingrese la longitud de la contraseña (mínimo 8): "))
while longitud < 8:
    print("¡Ingrese una longitud mayor o igual a 8 y menor de 64!")
    longitud = int(input("Ingrese longitud: "))
```

Complicación 1: limitaciones de RAPTOR al elaborar el diagrama de flujo

Al momento de diseñar el diagrama de flujo en RAPTOR Avalonia, se identificó que la herramienta no acepta ciertos comandos ni funciones de alto nivel que Python sí permite, como la generación directa de cadenas aleatorias en una sola instrucción. Esto obligó a replantear el diagrama de una forma más detallada y explícita, mostrando uno por uno los caracteres que conforman cada grupo (mayúsculas, minúsculas, números y símbolos) y el proceso de selección aleatoria de manera individual.

Aunque esta limitación representó un esfuerzo adicional, el resultado fue positivo: el diagrama resultante es más claro y didáctico, ya que muestra con mayor precisión lo que ocurre dentro del algoritmo en cada paso, algo que una instrucción compacta no habría evidenciado.

Complicación 2: almacenamiento de caracteres con variables de texto

En la primera versión del programa Python, los grupos de caracteres (letras mayúsculas, minúsculas, números y símbolos) se almacenaron directamente en variables de tipo cadena de texto (string). Esta decisión se tomó porque en ese momento los temas de listas y tuplas aún no habían sido abordados con profundidad en el curso, y la concatenación de cadenas resultaba más intuitiva y familiar.

La solución funcionó correctamente: al ser las cadenas de texto iterables en Python, era posible acceder a cualquier posición mediante un índice (conjunto[indice]) y conocer su longitud con len(conjunto), que es exactamente lo que el algoritmo de generación necesita. Sin embargo, esta implementación tenía margen de mejora en términos de organización y legibilidad del código.

```
#Primera versión, caracteres almacenados en strings
conjunto = ""
if usar_mayusculas == 1:
    conjunto = conjunto + "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

if usar_minusculas == 1:
    conjunto = conjunto + "abcdefghijklmnopqrstuvwxyz"

if usar_numeros == 1:
```

```
conjunto = conjunto + "0123456789"

if usar_simbolos == 1:
    conjunto = conjunto + "!@#$%^&*"
```

La mejora: uso de Tuplas gracias a los contenidos de la última clase

Al recibir la última clase del curso, cuyo contenido incluyó las estructuras de datos Tuplas y Listas, se comprendió que estas ofrecen una forma más organizada y semánticamente correcta de agrupar colecciones de elementos fijos. Las tuplas en particular son ideales para almacenar grupos de caracteres que no van a cambiar durante la ejecución del programa, ya que son inmutables y más eficientes en memoria.

Con esta nueva comprensión, el código fue mejorado para almacenar cada grupo de caracteres en una tupla, y luego combinarlos en un único conjunto mediante concatenación. Esto hizo el programa más legible, estructurado y alineado con las buenas prácticas de Python:

```
# Tuplas con los grupos de caracteres
mayusculas = ("A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M",
              "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z")

minusculas = ("a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m",
              "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z")

numeros = ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9")

simbolos = ("!", "@", "#", "$", "%", "^", "&", "*")
```

```
# Armar el conjunto como tupla sumando las tuplas seleccionadas
conjunto = ()
if usar_mayusculas == 1:
    conjunto = conjunto + mayusculas
if usar_minusculas == 1:
    conjunto = conjunto + minusculas
if usar_numeros == 1:
    conjunto = conjunto + numeros
if usar_simbolos == 1:
    conjunto = conjunto + simbolos
print("La contraseña se generará a partir de este contenido:", conjunto)
```

Reflexión sobre las complicaciones

Ambas complicaciones —las limitaciones de RAPTOR y el manejo inicial de los caracteres con strings— no fueron obstáculos sino parte natural del proceso de aprendizaje progresivo

en programación. La primera obligó a pensar el algoritmo con mayor detalle; la segunda condujo a una solución funcional que luego pudo mejorarse al adquirir nuevos conocimientos.

Esta evolución del código —de variables de texto a tuplas— ilustra un principio fundamental del desarrollo de software: el código no nace perfecto, sino que mejora con la experiencia y el aprendizaje. La última clase del curso no solo aportó un concepto nuevo, sino que generó una comprensión aplicada que se tradujo directamente en una mejora concreta del programa.

5. Conclusiones

- El Generador Seguro de Contraseñas desarrollado en Python resuelve una necesidad real de seguridad digital, demostrando que soluciones informáticas sencillas pueden tener un impacto concreto y positivo en el día a día de las personas.
- El proyecto integró los cuatro contenidos del curso: diseño de diagramas funcionales y de arquitectura, estructuras lógicas condicionales y repetitivas, organización modular del código y uso de GitHub como herramienta de control de versiones.
- La decisión de implementar el conjunto de caracteres como una variable de tipo string, en lugar de una lista o tupla, fue una solución técnica válida que permitió completar el proyecto usando los conocimientos disponibles, sin sacrificar la correcta funcionalidad del sistema.
- Las complicaciones encontradas durante el desarrollo, especialmente con el manejo de estructuras de datos, fueron oportunidades reales de aprendizaje que fortalecieron la comprensión de cómo los datos se representan y manipulan en Python.
- El proceso de planificación y diseño previo a la codificación fue determinante: haber definido los requerimientos, elaborado los diagramas y estructurado la arquitectura antes de programar permitió escribir un código más organizado, claro y libre de errores lógicos fundamentales.